

TP de Science de Données

Implémentation de l'algorithme des Nuées Dynamiques (K-Means) from
scratch

Étudiant

Table des matières

1	Introduction	2
2	Présentation théorique de l'algorithme	2
2.1	Principe général	2
2.2	Étapes de l'algorithme	2
2.3	Complexité algorithmique	2
3	Implémentation en Python	3
3.1	Code Source	3
3.2	Explication détaillée des fonctions clés	5
4	Résultats et Visualisation	5
4.1	Génération des données	5
4.2	Déroulement de l'algorithme	5
4.3	Visualisation	6
4.4	Analyse des résultats	6
5	Discussion et limites	6
5.1	Avantages de l'implémentation	6
5.2	Limites identifiées	6
5.3	Perspectives d'amélioration	6
6	Conclusion	7

1 Introduction

Dans le cadre du cours de Science de Données, ce travail pratique a pour objectif d'implémenter l'algorithme des **Nuées Dynamiques** (communément appelé *K-Means*) sans utiliser de bibliothèques de haut niveau comme `scikit-learn`. L'objectif est double : comprendre les mécanismes internes de l'algorithme et acquérir une maîtrise pratique des outils de calcul numérique en Python (`numpy`, `matplotlib`).

Le clustering est une tâche fondamentale en apprentissage non supervisé. Elle consiste à regrouper des objets similaires en classes homogènes sans connaissance préalable des étiquettes. L'algorithme des Nuées Dynamiques, introduit par MacQueen en 1967, est l'une des méthodes de partitionnement les plus utilisées en raison de sa simplicité et de son efficacité.

2 Présentation théorique de l'algorithme

2.1 Principe général

L'algorithme des Nuées Dynamiques est une méthode de partitionnement de données (clustering) non supervisée. Il vise à partitionner n observations en k groupes (clusters) homogènes. Le principe repose sur une minimisation de l'inertie intra-classe, c'est-à-dire la somme des distances au carré entre chaque point et le centre de son cluster.

Formellement, on cherche à minimiser la fonction objectif suivante :

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

où C_i est le i -ème cluster et μ_i son centre de gravité.

2.2 Étapes de l'algorithme

L'implémentation suit un processus itératif en 4 étapes :

1. **Initialisation** : Choix aléatoire de k points comme centres initiaux.
2. **Assignment** : Chaque point est assigné au centre le plus proche (distance euclidienne).
3. **Mise à jour** : Les centres sont recalculés comme la moyenne des points de leur cluster.
4. **Convergence** : L'algorithme s'arrête lorsque le déplacement des centres est inférieur à un seuil de tolérance, ou après un nombre maximum d'itérations.

2.3 Complexité algorithmique

La complexité temporelle de l'algorithme est de l'ordre de $O(n \cdot k \cdot d \cdot t)$, où n est le nombre de points, k le nombre de clusters, d la dimension des données et t le nombre d'itérations. Cette complexité reste raisonnable pour des jeux de données de taille modérée.

3 Implémentation en Python

Le code ci-dessous présente l'implémentation complète réalisée « from scratch » à l'aide de la bibliothèque numpy.

3.1 Code Source

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # 1. FONCTION DE DISTANCE EUCLIDIENNE
5 def distance_euclidienne(point, centre):
6     """Calcule la distance euclidienne entre un point et un centre."""
7     return np.sqrt(np.sum((point - centre) ** 2))
8
9 # 2. INITIALISATION DES CENTRES (aleatoire)
10 def initialiser_centres(X, k):
11     """Choisit k points au hasard dans X comme centres initiaux."""
12     indices = np.random.choice(len(X), k, replace=False)
13     return X[indices].copy()
14
15 # 3. ASSIGNATION DES POINTS AUX CLUSTERS
16 def assigner_clusters(X, centres):
17     """Assigne chaque point au centre le plus proche."""
18     labels = []
19     for point in X:
20         distances = [distance_euclidienne(point, centre) for centre in
21                     centres]
22         labels.append(np.argmin(distances))
23     return np.array(labels)
24
25 # 4. MISE A JOUR DES CENTRES
26 def mettre_a_jour_centres(X, labels, k):
27     """Recalcule chaque centre comme la moyenne des points assignes."""
28     nouveaux_centres = np.zeros((k, X.shape[1]))
29     for i in range(k):
30         points_du_cluster = X[labels == i]
31         if len(points_du_cluster) > 0:
32             nouveaux_centres[i] = points_du_cluster.mean(axis=0)
33         else:
34             # Si un cluster est vide, on le reinitialise au hasard
35             nouveaux_centres[i] = X[np.random.choice(len(X))]
36     return nouveaux_centres
37
38 # 5. ALGORITHME PRINCIPAL : NUEES DYNAMIQUES
39 def nuees_dynamiques(X, k, max_iter=100, tolerance=1e-4):
40     """
41     Algorithme des Nuees Dynamiques (K-Means) from scratch.
42
43     Parametres :
44         X : array (n_samples, n_features) -> les donnees
45         k : int -> nombre de clusters
46         max_iter : int -> nombre maximum d'iterations
47         tolerance : float -> seuil de convergence
48
49     Retourne :
50         centres : array (k, n_features)
```

```

50     labels : array (n_samples,)
51     """
52     # 1. Etape d'Initialisation
53     centres = initialiser_centres(X, k)
54
55     for iteration in range(max_iter):
56         # 2. Assignment
57         labels = assigner_clusters(X, centres)
58
59         # 3. Mise a jour
60         anciens_centres = centres.copy()
61         centres = mettre_a_jour_centres(X, labels, k)
62
63         # 4. Test de convergence
64         deplacement = np.sum(np.sqrt(np.sum((centres - anciens_centres)
65 ** 2, axis=1)))
66         print(f"Iteration {iteration + 1} - Deplacement total des
67 centres : {deplacement:.6f}")
68
69         if deplacement < tolerance:
70             print(f"Convergence atteinte a l'iteration {iteration + 1}")
71             break
72
73     return centres, labels
74
75 # 6. GENERATION DE DONNEES DE TEST
76 def generer_donnees(n_points=300, k=3):
77     """Genere k groupes de points autour de centres aleatoires."""
78     np.random.seed(42)
79     X = []
80     for i in range(k):
81         centre = np.random.rand(2) * 10
82         points = np.random.randn(n_points // k, 2) + centre
83         X.append(points)
84     return np.vstack(X)
85
86 # 7. PROGRAMME PRINCIPAL
87 if __name__ == "__main__":
88     # Generation des donnees
89     X = generer_donnees(n_points=300, k=3)
90     print(f"Donnees generees : {X.shape[0]} points en {X.shape[1]}
91 dimensions\n")
92
93     # Lancement de l'algorithme
94     k = 3
95     centres, labels = nuees_dynamiques(X, k, max_iter=100, tolerance=1e
96 -4)
97
98     print("\n--- Resultats ---")
99     print(f"Centres finaux :\n{centres}")
100     print(f"Nombre de points par cluster : {np.bincount(labels)}")
101
102     # Visualisation
103     plt.figure(figsize=(10, 6))
104     couleurs = ['red', 'blue', 'green', 'orange', 'purple']
105
106     for i in range(k):
107         points_cluster = X[labels == i]

```

```

104     plt.scatter(points_cluster[:, 0], points_cluster[:, 1],
105                c=couleurs[i % len(couleurs)], label=f'Cluster {i}',
106                alpha=0.6)
107
108     plt.scatter(centres[:, 0], centres[:, 1],
109                c='black', marker='X', s=300, edgecolors='yellow',
110                linewidths=2, label='Centres')
111
112     plt.title("Nuees Dynamiques (K-Means) - Implementation from scratch"
113             )
114     plt.xlabel("X1")
115     plt.ylabel("X2")
116     plt.legend()
117     plt.grid(True, alpha=0.3)
118     plt.show()

```

Listing 1 – Implémentation des Nuées Dynamiques

3.2 Explication détaillée des fonctions clés

- `distance_euclidienne` : Calcule la distance entre un point et un centre via la formule $\sqrt{\sum (x_i - c_i)^2}$. Cette fonction est appelée pour chaque point et chaque centre à chaque itération.
- `initialiser_centres` : Utilise `np.random.choice` pour sélectionner k indices uniques dans le jeu de données. Cette approche est simple mais sensible aux minima locaux.
- `assigner_clusters` : Parcourt chaque point et calcule sa distance à tous les centres pour trouver le minimum (`np.argmin`). C'est l'étape la plus coûteuse en temps.
- `mettre_a_jour_centres` : Calcule la moyenne des points appartenant à chaque cluster. Une gestion des clusters vides est prévue en réinitialisant le centre aléatoirement, ce qui évite les erreurs de division par zéro.
- `nuees_dynamiques` : Orchestre le processus itératif et vérifie la convergence via le déplacement total des centres. Le critère d'arrêt combine un nombre maximum d'itérations et un seuil de tolérance.

4 Résultats et Visualisation

Pour tester l'algorithme, nous avons généré un jeu de données synthétique composé de 300 points répartis en 3 groupes (clusters) distincts.

4.1 Génération des données

Les données sont générées avec une distribution gaussienne autour de centres aléatoires. Chaque cluster contient 100 points. L'utilisation de la graine aléatoire (`np.random.seed(42)`) garantit la reproductibilité des résultats.

4.2 Déroulement de l'algorithme

L'algorithme converge généralement en moins de 20 itérations pour ce jeu de données. Le déplacement total des centres diminue progressivement jusqu'à devenir inférieur au seuil de tolérance (10^{-4}).

4.3 Visualisation

La figure générée (voir Figure 1) montre la répartition finale des points et la position des centres (marqués d'une croix noire). On observe une séparation nette des trois clusters, ce qui confirme le bon fonctionnement de l'algorithme.

FIGURE 1 – Visualisation des clusters après convergence de l'algorithme.

4.4 Analyse des résultats

Les centres finaux obtenus sont proches des centres réels utilisés lors de la génération des données. Le nombre de points par cluster est équilibré (environ 100 points par cluster), ce qui est cohérent avec la construction du jeu de données.

5 Discussion et limites

5.1 Avantages de l'implémentation

- **Transparence** : Chaque étape de l'algorithme est explicite et contrôlable.
- **Pédagogique** : Permet de comprendre les rouages internes du K-Means.
- **Légèreté** : Aucune dépendance à des bibliothèques lourdes comme `scikit-learn`.

5.2 Limites identifiées

- **Sensibilité aux centres initiaux** : L'algorithme peut converger vers un minimum local. Une initialisation aléatoire peut donner des résultats différents à chaque exécution.
- **Complexité** : La boucle explicite en Python pour l'assignation est lente pour de grands jeux de données. Une vectorisation avec `numpy` améliorerait les performances.
- **Choix de k** : La méthode ne détermine pas automatiquement le nombre optimal de clusters. Il faut utiliser des heuristiques comme la méthode du coude ou le coefficient de silhouette.
- **Forme des clusters** : L'algorithme suppose des clusters sphériques et de taille similaire, ce qui n'est pas toujours adapté aux données réelles.
- **Gestion des outliers** : Les points aberrants peuvent influencer de manière significative la position des centres.

5.3 Perspectives d'amélioration

Pour aller plus loin, plusieurs pistes peuvent être envisagées :

- Implémenter l'initialisation *K-Means++* pour réduire la sensibilité aux centres initiaux.
- Utiliser la méthode du coude (*elbow method*) pour choisir k .
- Vectoriser les calculs de distance avec `numpy` pour améliorer les performances.
- Comparer les résultats avec l'implémentation de `scikit-learn`.
- Tester l'algorithme sur des jeux de données réels (Iris, Wine, etc.).

6 Conclusion

Ce TP a permis de mettre en œuvre l'algorithme des Nuées Dynamiques de manière exhaustive. L'implémentation from scratch a mis en évidence l'importance du critère de convergence et de l'initialisation aléatoire. Les résultats montrent que l'algorithme converge rapidement vers une partition stable des données.

Au-delà de la simple implémentation, ce travail a permis de mieux comprendre les enjeux du clustering non supervisé et les limites des méthodes de partitionnement. L'algorithme des Nuées Dynamiques reste un outil puissant et largement utilisé, mais il doit être employé avec discernement en fonction de la nature des données.

Références

- MacQueen, J. (1967). *Some methods for classification and analysis of multivariate observations*. Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability.
- Jain, A. K. (2010). *Data clustering : 50 years beyond K-means*. Pattern Recognition Letters.
- Documentation officielle de `numpy` : <https://numpy.org/doc/>